

PRACTICAL-3

Aim: Implementation of max-heap sort algorithm

Algorithm:

Max Heap sort:

Step 1: Start with an unsorted array A of elements

Step 2: Build a max Heap from the array.
→ start from the last non-leaf node
 $i = (n/2) - 1$
→ Apply max Heapify for every node up to root.

Step 3: The largest element is now at the root A[0].

Step 4: Swap the root element A[0] with the last element A[n-1]

Step 5: Reduce the heap size by 1

Step 6: Apply max Heapify to the root to restore the max Heap property.

Step 7: Repeat steps 4-6 until the heap size becomes 1

Step 8: The array is now sorted in ascending order.

Step 9: Stop if and stop for.

Program:

```
#include <iostream>

using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
```

```
        largest = right;
    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

```
int arr[] = {12, 11, 13, 5, 6, 7};  
int n = sizeof(arr) / sizeof(arr[0]);  
cout << "Original array: ";  
printArray(arr, n);  
heapSort(arr, n);  
cout << "Sorted array: ";  
printArray(arr, n);  
return 0;  
}
```

Output:

```
Original array: 12 11 13 5 6 7  
Sorted array: 5 6 7 11 12 13  
  
=== Code Execution Successful ===
```

Time complexity:

for (int i = n/2 - 1; i >= 0; i--) $\rightarrow n/2$
 Heapify(arr, n, i) $\rightarrow \log n$
 for (int i = n - 1; i > 0; i--) $\rightarrow n - 1$
 {
 swap(arr[0], arr[i]) $\rightarrow 1$
 Heapify(arr, i + 1, 0) $\rightarrow \log n$
 }
 $T(n) = n/2 \log n + (n - 1)(1 + \log n)$
 $T(n) = n/2 \log n + (n - 1) + (n - 1) \log n$
 $T(n) = n/2 \log n + n \log n + n - 1 - \log n$
 $T(n) = O(n \log n)$
 Best case: $O(n \log n)$
 Worst case: $O(n \log n)$
 Average case: $O(n \log n)$

Conclusion:

The Max-Heap Sort algorithm was successfully implemented in C++. The algorithm first constructs a Max Heap and then repeatedly extracts the maximum element to sort the array in ascending order. Heap Sort has a time complexity of $O(n \log n)$ in the best, average, and worst cases, making it an efficient sorting algorithm.